

# 스토리라인

☰ 진행상황

진행중

## 1. 기 — 우리는 이 문제를 어떻게 마주했는가

자동차가 바뀌고 있다. 예전 차는 엔진·브레이크 같은 하드웨어가 고장 났지만, 지금 차는 소프트웨어로 굴러간다. SDV(소프트웨어 정의 차량)로 넘어가면서 기능 대부분이 코드로 돌아가고, OTA로 출고된 뒤에도 계속 업데이트된다. 그런데 여기서 새로운 종류의 결함이 생긴다 — 하드웨어처럼 눈에 보이지 않고, 출고 후에도 새로 유입되는 SW 결함이다.

그 대표 사례가 ICCU 사태다. 현대·기아 전기차 55만 대가 리콜됐는데, 문제는 리콜 이후에도 같은 증상 신고가 계속 올라왔다는 거다. 즉 한 번 리콜했다고 끝이 아니라, 결함 신호가 리콜 전후로 계속 흐르고 있었다. 이건 "한 번의 리콜"로 해결되는 문제가 아니라, 신호를 계속 추적·감시해야 하는 문제였다.

그런데 지금 이걸 누가 보고 있나? 미국 NHTSA에서는 사람이 월 6천 건이 넘는 소비자 불만(VOQ)을 직접 읽으며 "이게 조사할 만한 결함인가"를 판단한다. 사람이 읽는 이상 병목이 생기고, 리콜 전후의 반복 신호를 놓친다. **우리가 마주한 문제는 여기였다 — 결함 신호는 이미 텍스트로 쌓이고 있는데, 그걸 제때 읽어내는 사람이 부족하다.**

## 2. 승 — 그래서 우리는 이걸 어떻게 풀려고 했는가

우리는 "사람을 대체하겠다"가 아니라 **"사람이 조사에 착수하기 전에, 우선순위·근거·상태를 정리해주는 검토 보조 AI Agent"**를 만들기로 했다. 사람이 6천 건을 다 읽는 대신, Agent가 먼저 급증 신호를 찾아 근거와 함께 정리해주면, 사람은 그 위에서 판단만 하면 된다.

여기서 "왜 그냥 챗봇이나 RAG가 아니라 Agent인가"라는 질문이 나온다. 챗봇은 한 번 물으면 한 번 답하고 끝이다. RAG는 검색해서 요약하는 고정된 경로다. 그런데 결함 조사는 그렇게 단순하지 않다 — 가설을 세우고, 데이터로 확인하고, 아니면 다시 세우고, 근거가 부족하면 보류하는 반복 과정이다. 그래서 우리는 질문에 따라 조사 경로가 바뀌고, 스스로 가설을 검증하는 Agent 구조를 택했다. (LLM은 이 안에서 판단과 서술을 맡을 뿐, 계산과 검증은 코드가 한다 — 이 역할 분리가 뒤에서 다시 나온다.)

그리고 이 Agent를 어떻게 "제대로 작동한다"고 증명할 것인가. 여기서 우리 팀의 핵심 설계가 나온다 — **입력 데이터와 정답지를 시간으로 분리했다**. 리콜이 발표되기 이전 데이터만 Agent에게 주고, "실제 리콜보다 먼저 이 신호를 잡아내는가"를 채점한다. 리콜은 이미 일

어난 역사적 사실이니까, 그걸 정답지로 두면 "우리 Agent가 정말 조기에 경고할 수 있는가"를 객관적으로 검증할 수 있다. 이게 우리가 K12 퇴감기 평가라고 부르는 방식이다.

### 3. 전 — 만들면서 무엇에 부딪혔고, 어떻게 파고들어 풀었는가

우리 프로젝트의 진짜 이야기는 여기서다. 화려한 기술이 아니라, 부딪히는 문제마다 파고들어 풀어낸 과정이다.

#### (1) 구조화 — 자유 텍스트를 조사 가능하게, 그런데 AI가 지어내면?

소비자 불만은 제각각의 문장이다. 부품·증상·심각도로 구조화하는데, "AI가 근거를 지어내면 어쩌나"에 부딪혔다. evidence\_quote가 원문에 실제로 있는지 자동 대조하는 검사를 넣었고, 100건에서 인용 불일치 0건. 그 과정에서 원본 CSV의 인코딩 깨짐(모지바케)이 "가짜 환각"으로 잡히는 걸 발견해, 진짜 환각과 인코딩 문제를 분리하는 복구 로직까지 넣었다.

#### (2) 검증 — 정답이 없는데 어떻게 채점하나 (순환 논리와 싸움)

가장 어려운 문제였다. 이 데이터엔 정답 라벨이 없다. 다른 AI로 채점하면? "AI가 AI를 채점"하는 순환 논리다. 내가 만든 규칙으로 채점하면? "내 규칙으로 내가 채점"하는 순환 논리다. 우리는 이 방법들을 하나씩 기각하며 좁혀갔고, 결국 "같은 모델을 여러 번 돌려 스스로 얼마나 일관되게 판단하는지"만 측정하는 방법에 도달했다 — 남과 비교하지 않으니 순환 논리에서 자유롭다. 그리고 이 수치를 절대적 품질로 과대 해석하지 않고 "어디를 사람이 먼저 봐야 하는가"라는 상대적 우선순위로만 쓰기로 선을 그었다.

#### (3) 조사 — AI에게 코드를 맡기지 않는다

조사 루프에서 AI에게 자유롭게 판다스 코드를 짜게 하면 버그·보안·비결정성을 통제할 수 없다. 그래서 미리 검증한 조회 함수 8개를 만들고, AI는 그중 "이름만" 고르게 했다. 계산은 코드가, 판단만 AI가. 그리고 "가설을 세우기 전에 무조건 실제 숫자부터 본다", "근거가 부족하면 RETAINED/REJECTED가 아니라 HOLD로 보류한다"는 원칙을 넣었다.

#### (4) 실전 — mock에서 진짜 데이터로 옮기니 다 터졌다

구조가 돌아가는 걸 mock으로 확인한 뒤 실제 데이터 16,964건으로 옮기자 리콜 매칭이 0건이 됐다. 추적해보니 데이터 간 ID가 안 맞았고, Git에서 진짜 원본을 발굴해 교체했다. 그러자 이번엔 EV9·아이오닉6 등 26개 차종이 제조사 UNKNOWN으로 누락되는 버그가 드러났다. 직접 매핑을 채워 해소했고, 그제서야 EV9 계기판 리콜(24V757000)이 실제 신고 8건과 정확히 매칭되며 "리콜 후 재발" 상태가 실사례로 처음 증명됐다.

#### (5) 공급사 — 그럴듯함을 믿지 않는다 ← 추가된 부분! 아래서 자세히

#### (6) 정직 — 없으면 없다고 말한다

가짜 급증 대조군(GENESIS)에서 신호가 떴을 때, 억지로 결함을 만들지 않고 "일관된 새 신호 없음"으로 보고했다. 알고 보니 그건 기존 리콜에 대한 후속 항의였고, 우리는 이런 "기존 리콜 언급"을 자동 판별하는 필드를 따로 만들어 신규 신호와 분리했다.

이 여섯을 관통하는 하나 — **검증되기 전엔 말하지 않았고, 틀렸으면 되돌렸다.**

## **+5번 공급사 파트 추가 설명**

### *(1) 왜 했나 — "신고 29건 급증"의 한계*

이때까지 우리 시스템이 할 수 있는 최선은 "이 차종에 신고가 몇 건 급증했다"까지였다. 그런데 실제로 상담사나 조사관이 알아야 할 건 "그래서 어느 부품이, 누가 만든 게, 왜 문제인가"다. 이걸 채우려면 NHTSA 리콜 공식 문서(Part 573 리포트)에서 부품명·부품번호·공급사·공식 결함원인을 추출해야 했다. 이게 부품 지식 레이어다.

### *(2) 뭘 했나 — 76건 수집 → 정규화 → DB → 화면*

SW 관련 리콜 76건의 Part 573 문서를 수집해 부품 정보를 구조화했다(198행). 대표 사례로 EV9 계기판은 부품번호 940C3-DO010, 현대모비스 제조로 확인됐다. 이걸 webapp DB의 parts 테이블에 넣어, 채팅 답변과 내 차 화면이 "공식 결함원인 원문 + 부품번호 + 공급사"까지 근거로 보여주게 했다.

### *(3) 어디서 막혔나 — 공급사 이름이 지저분했다*

같은 회사가 여러 표기로 흩어져 있었다(Hyundai Mobis / MOBIS / 대소문자 변형 등 47종). 이걸 정규화하지 않으면 나중에 "공급사별 집계"를 할 때 같은 회사가 여러 줄로 쪼개진다. 그래서 표기를 정리해 대표명(canonical)과 그룹(supplier\_group) 2층 구조로 묶었다.

### *(4) 어떻게 풀었나 — 그럴듯함을 믿지 않고 원문을 봤다 (★핵심)*

여기서 중요한 판단이 있었다. 정규화하다 보니 "Seoyon E-Hwa Alabama"와 "SEOYON E-HWA AUBURN"이 같은 공장처럼 보였다. 오번(Auburn)이 앨라배마주 도시니까 자연스럽게 병합하려 했다. 그런데 원문 573 문서의 주소를 직접 대조하니 — 하나는 Montgomery, 하나는 Dadeville로 실제로 다른 사업장이었다. 그래서 병합을 철회했다. 반대로 Transys Korea는 현대트랜시스로 추정됐지만 원문에 "Hyundai" 표기가 없어 그룹 확정을 보류했다. **그렇듯해 보여도 원문으로 확인되기 전엔 확정하지 않는다** — 이게 우리 팀 전체가 지킨 원칙과 정확히 같다.

### *(5) 왜 중요한가 — 세 가지 확장의 근거*

이 부품 데이터가 채워지자 세 가지가 가능해졌다.

① 채팅 답변이 "신고 급증"에서 "공식 결함원인·부품·공급사"로 격상됐다.

② 같은 부품 계열(940C3)이 12개 차종에 걸친다는 게 데이터로 드러나, "공유 부품 결함이 여러 차종으로 전파된다"는 확장 슬라이드의 실증이 됐다.

③ 공급사 집계 결과 현대모비스가 198건 중 123건(62%)으로, "전장·SW 결함이 특정 공급망에 집중된다"는 발표 포인트가 나왔다.

#### 4. 결 — 그래서 무엇이 만들어졌고, 어디까지 가는가

이 과정을 거쳐 실제로 작동하는 서비스가 나왔다. 상황판에서 급증을 감지하고, 조사 채팅에서 근거와 함께 답하고, 내 차 화면에서 도메인별 리콜을 보여준다. 특히 최근에는 부품 지식 레이어를 더해, "신고 29건 급증"에서 "이 결함의 공식 원인은 이것, 부품은 940C3 계열, 공급사는 현대모비스"까지 근거를 내려줄 수 있게 됐다. (여기서 라이브 데모.)

그리고 이 데이터는 두 방향으로 쓰인다. 소비자에게는 조사 채팅으로, 상담사에게는 같은 엔진으로 "고객 안내 요약"을 즉시 제공한다 — 새 기능을 만든 게 아니라 같은 조사 엔진에 표시만 다르게 했다.

더 멀리 보면, 이 결함 지식은 설계 단계로 되돌아갈 수 있다. 같은 부품 계열이 여러 차종에서 반복 결함을 낸다는 걸 우리 데이터가 이미 보여줬다(940C3이 12차종). 지금은 결함을 사후에 감지하지만, 다음은 신차 설계 때 "이 부품 조합은 과거에 문제가 있었다"고 미리 알려주는 기획 피드백으로 확장된다. 결함 탐지에서 → 부품 지식 축적으로 → 설계 피드백으로. 이게 MOBISCOPE의 로드맵이다.